



Shri Vile Parle Kelavani Mandal's
INSTITUTE OF TECHNOLOGY
DHULE (M.S.)
DEPARTMENT OF COMPUTER ENGINEERING

Subject : Machine Learning Lab Name: _____ Roll No. : _____ Class: T. Y. Comp. Engg. Batch : T Division: _____ Expt. No. : 01 Date : / / Title : Write a program to Perform Exploratory Data Analysis and Data Visualization in Python.	Remark Signature
--	---

Rubrics :	<table border="1" style="width: 100%; border-collapse: collapse;"> <tr> <th colspan="3" style="text-align: center;">Date of Submission:</th> </tr> <tr> <th style="width: 33%;">Marks Split up to</th> <th style="width: 33%;">Maximum Marks</th> <th style="width: 33%;">Marks Obtained</th> </tr> <tr> <td>Performance/conduction</td> <td></td> <td></td> </tr> <tr> <td>Report Writing</td> <td></td> <td></td> </tr> <tr> <td>Attendance</td> <td></td> <td></td> </tr> <tr> <td>Viva/Oral</td> <td></td> <td></td> </tr> <tr> <td>Total Marks</td> <td></td> <td></td> </tr> </table>	Date of Submission:			Marks Split up to	Maximum Marks	Marks Obtained	Performance/conduction			Report Writing			Attendance			Viva/Oral			Total Marks		
Date of Submission:																						
Marks Split up to	Maximum Marks	Marks Obtained																				
Performance/conduction																						
Report Writing																						
Attendance																						
Viva/Oral																						
Total Marks																						
Aim:	The main aim of this lab is to demonstrate the methods used or data analysis and visualization from the Pandas, Seaborn and Matplotlib library of Python. The secondary objectives are: 1. to explain how a dataset is loaded as Pandas data frame, 2. how the required information is extracted from the data frame, and 3. how the dataset is visualized through different kinds of graphs.																					
Software Required :	Jupyter Notebook , Python 3.6, Or Google Colab																					
Dataset :	Titanic.csv																					

Theory :**Dataset initialization:**

The dataset initialization can be divided into two parts. First is importing necessary modules. Here, we import Pandas, Matplotlib and Seaborn modules.

The second activity in the data initialization includes loading of the dataset into memory for processing. Here, we are assuming a structured and cross-sectional dataset. The structured dataset is usually available in CSV (Comma Separated Variables) format. By cross-sectional, I mean data instances are collected over the period of a single duration.

Python's Pandas library provides the method `read_csv()` to read a CSV file and to load it as a data frame. The data frame is two-dimensional data structures like the matrix. It can be indexed through non-integer variables also. Further, the data frame method `head()` displays complete information of the first five instances in the dataset which is also referred to as the header

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

```
df=pd.read_csv('../input/titanicdataset-traincsv/train.csv')
df.head()
```

The next activity is to get the meta-information about the dataset. The attribute `shape` and `columns` provide information about the dimensional size of the dataset and names of features or dataset variables.

The method `info()` provides information about feature name, feature type, and whether there are any missing values are present for any of the features.

```
print(df.shape)

df.columns

print(df.info())
```

The descriptive statistics of numeric variables is obtained through the data frame method `describe()`. For each numeric data feature, this method displays the total number of instances (count), mean, standard deviation (std), minimum value (min), first quartile (25%), third quartile (75%) and the maximum value.

This method is quite useful to get the initial insight into the dataset.

In case the parameter `'include="object"'` is specified, the method `describe()` displays the descriptive statistics of categorical or discrete type of variables. For categorical variables, this method shows information such as a specific number of values for a given categorical variable, the value occurring most of the times in the data set, and its count.

```
df.describe()

df.describe(include='object')
```

The method **value_counts()** which is invoked on a specific column of a data frame is another useful method to count the occurrences of individual values for a given feature variable. The parameter 'normalize=True' can be used to get a percentage of occurrences of individual value in a specified column.

```
df['Sex'].value_counts()
```

```
df['Sex'].value_counts(normalize=True)
```

Summary Table 1

Task	Command
To load a CSV file	df= pd.read_csv()
To Print the header of CSV file	df.head()
To print dimensional size of the data set	df.shape()
To print names of variables or features	df.columns()
To print types of variables or features	df.info()
To print descriptive statistics of numeric variables	df.describe()
To print descriptive statistics of non-numeric variables	df.describe(include= 'object')
To print counts of distinct values for categorical variables	df['Sex'].value_counts()
To print percentage of distinct values for categorical variables	df['Sex'].value_counts(normalize=True)

Querying a dataset or DataFrame

- Querying a dataset or DataFrame with Pandas Pandas provides several methods to query a particular dataframe.
- The query methods provided in Pandas are as expressive as statements in any query languages (e.g., SQL) as supported by relational databases.
- This section illustrates the application of query mechanisms for the given Titanic dataset. Projecting columns or feature variables
- There are multiple ways to access a particular column or feature variable in Pandas.
- The first and simplest way is to index the dataframe with the name of the desired column.
- For example, *df['Name']* returns all the values in the 'Name' column as a Pandas's series.
- Further, we can invoke the methods such as head(), mean() and max() on the column values returned as series.

```
df['Name'].head()
```

The second way of projecting columns and instances is to use loc[] and iloc[] methods. The loc[] method accesses columns with column name as indexes. At the same time, iloc[] uses integers as an index which indicate the column position—use of square brackets[] to be noted here.

```
df.loc[0:15, 'Name':'Age']
```

```
df.iloc[0:15,3:6]
```

More complex queries can be constructed to extract rows or data instances, satisfying a particular condition—for example, printing name of the oldest passenger. Here, we need to first find the largest values for the age and print the data instances satisfying the condition `age == max age`.

```
df[df['Age']== df[df['Sex'] =='male']['Age'].max()]['Name']
```

Sorting Values in a Column: The method `sort_values()` can be used to sort the values in a dataframe's column. It takes two arguments. The first is the name of a column to be sorted (by) and the second is the order of sorting. (ascending). Here are a few examples.

```
df.sort_values(by='Name').head()
```

```
df.sort_values(by=['Name', 'Age'], ascending=[True, False]).head()
```

Replacing values in a column:

Sometimes, we need to replace all values in a column. Such situations arise when we want to convert all categorical values into supposing numeric values. Pandas provides two convenient ways to achieve this task. The first is through the application of `map()` method and the second is through the `replace()` method.

The following example replaces the values 'male' and 'female' in the 'Sex' column with values '0' and '1'. Here we define a new dictionary with old values of the column as keys (male, female) and new values as the value (0,1) in the dictionary. Then this dictionary is passed as an argument to the `map()` method.

```
d={'male':0, 'female':1}
df['Sex'] = df['Sex'].map(d)
df.head()
```

The second way of replacing values in a column is through the method `replace()`. Like in the previous example, we need to define a dictionary. The newly created dictionary is included as a value and column name as the key in the dictionary to be passed as an argument to the `replace()` method.

The following example restores the values in the 'Sex' column back to 'male' and 'female' through `replace()` method.

```
o={0: 'male', 1: 'female'}
df= df.replace({'Sex':o})
df.head()
```

Grouping columns value-wise:

The method `groupby()` can be used to group the data instances according to the values of a categorical variable and print the descriptive statistics value-wise.

For example, the data instances in the Titanic data set are grouped for 'Survived' (a categorical variable) and an average 'Age' (a numerical variable) for survived and not survived passengers are calculated.

```
df.groupby(by='Survived')['Age'].describe()
```

Another way of grouping data instances or rows is through *crosstab()* method. It allows to group rows through multiple categorical variables.

The following example groups rows by 'Sex' and 'Survived' columns and presents genderwise percentage (normalize =True) of survived and not survived passengers.

```
pd.crosstab(df['Sex'],df['Survived'], normalize=True)
```

The method *pivot_table()* is the more flexible way of grouping rows. It allows to group rows through a categorical variable and presents the descriptive statistics for multiple numerical variables.

```
df.pivot_table(['Age', 'Fare'], ['Survived'], aggfunc='mean')
```

Summary Table 2

Task	Method
Projecting a column i.e. a feature	<code>df['fname']</code>
Selecting rows and columns	<code>Df.loc[0:5, 'fname':fname]</code> <code>df.iloc[0:5, 0:5]</code>
Sorting	<code>df.sort_values(by=[], ascending=[])</code>
Replacing values in a column	Map and replace method
Grouping values	<code>df.groupby().describe()</code>
Relating two or more than two variables	<code>df.crosstab()</code> <code>df.pivottable()</code>

Data Visualization

Python provides diverse ways to visualize dataset through libraries such as Pandas, Matplotlib and Seaborn.

The objective of this section is to familiarize with different methods of visualization and how different kinds of graphs such as linechart, barchart, histogram, and scatter plot are plotted.

Line Graph : The first and most straightforward way is to depict a data set through a line graph. The line graph is typically used for numerical and time-series data.

Here we use the *lineplot()* method from Seaborn to plot a graph age vs fare paid. Both are numeric data. In the line plot, two points are joined through lines.

```
sns.lineplot(x='Age', y='Fare', data=df)
```

Bar Graph : Python provides diverse ways to visualize the dataset. The second kind of graph that is frequently used to depict the data is the bar graph. The bar graphs are used to display counts of values for categorical or a discrete variable.

Here we use a bar graph to display counts of survived and not survived passengers as a bar graph through the *factorplot()* method from Seaborn library.

```
sns.factorplot(x='Survived', data=df, kind='count')
```

The *factorplot()* method can take an additional categorical variable through the 'hue' parameter. Here in the following example, we display the genderwise distribution of survived and not-survived passengers.

```
sns.factorplot(x='Survived', data=df, kind='count', hue='Sex')
```

Histograms : The histograms can be drawn through the *hist()* method from Pandas library. The *hist()* method takes the number of bins as one of the arguments. The histograms are useful to visualize the distribution of numeric of data. Here in this example, we display distribution of age through histograms.

From the histograms, it can be easily inferred that the majority of the passengers belong to age group 20-40years.

```
df['Age'].hist(bins=8)
```

Scatter plot : The scatter plot is used to observe how one variable relates to another variable. The *scatter()* method from Pandas library is used to draw the scatter plot.

The following examples plot two scatter plots. First is 'Age' vs 'Fare' and the second 'Age' vs 'Survived'.

```
df.plot.scatter(x='Age', y='Fare')
```

```
df.plot.scatter(x='Age', y='Survived')
```

Pie Chart : Plotting a pie chart is somewhat complicated in Python. Here we use the method *subplots()* from matplotlib and draw a pie chart to display value-wise composition for the 'Sex' variable.

```
import matplotlib.pyplot as plt
sizes= df['Survived'].value_counts()
fig1,ax1 = plt.subplots()
ax1.pie(sizes,labels=['Not Survived',
'Survived'],autopct='%1.1f%%',shadow=True)
plt.show()
```

Conclusion	
Test your Knowledge	In this way we successfully completed the Exploratory data analysis and visualization from the Pandas, Seaborn and Matplotlib library of Python. 1. Write a query to find the name of passenger whose age is minimum and gender female and write output 2. How many columns contains missing values. Write Name of Columns. 3. Write a code to replace categorical values in 'Embarked' column to Numerical 4. If you want to find the frequency Distribution of any numerical column which is the suitable chart/plot? 5. Differentiate between iloc and loc attribute of pandas dataframe

--	--